Nothing Ethereal About Ethereum

Copyright © 2023 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 Starting a Private Ethereum Blockchain Network **1**

2 Ethereum Account Exploration **2**

3 Ethereum Coin Transfer **5**

Overview

Ethereum is a decentralized, open-source blockchain with smart contract functionality. Ether (ETH) is the native cryptocurrency of the platform. Among cryptocurrencies, Ether is second only to Bitcoin in market capitalization. Ethereum network is a peer-to-peer network consists of multiple nodes running the Ethereum client such as Geth.

1 Starting a Private Ethereum Blockchain Network

Download the GitHub repository that contains the Docker configuration files for this lab.

```
git pull https://github.com/liranmatcu/Docker_Security_Labs.git
# If you have not downloaded the repository previously,
# you need to use git clone instead of pull
```

Start a private Ethereum network as follows:

```
cd Docker_Security_Labs/ethereum
docker-compose up -d
```

In this private network, there are three nodes: i) Bootnode — the bootstrap node that is used for peer discovery purpose. It listens on port 30303, the other nodes joining the network connect to this bootnode first. ii) JSON-RPC endpoint — this node exposes JSON-RPC API over HTTP endpoint on port 8545. iii) Miner — this node is responsible for mining (the process of creating a new block in our blockchain). When the miner node successfully mines a new block, it receives the rewards into the configured account. For more information, please check the docker-compose.yml file.

To setup a private Ethereum blockchain, the Ethereum client requires some information for creating the first block which is called the genesis block. A simple custom genesis.json containing information to create the genesis block is provided in the repository folder.

1.0.1 What is a genesis block? Why it is necessary? Please check the "genesis.json" for more details

Reference: https://arvanaghi.com/blog/explaining-the-genesis-block-in-ethereum/.

1.0.2 What is the meaning of chainId and ethash?

1.0.3 What is the meaning of ethash?

1.0.4 What is the meaning of proof-of-work? How to set it?

1.0.5 What is the meaning of difficulty value? How should it be adjusted?

2 Ethereum Account Exploration

After the containers are up and running, we can interacting with the blockchain via Inter-process Communication (IPC) or the RPC endpoint. We can use a software wallet, such as MetaMask, to connect to our private blockchain via the HTTP-RPC endpoint. Here we will use the **curl** command to demonstrate how we interact with our private blockchain.

We start by checking the connectivity of the nodes.

```
curl --location --request POST 'localhost:8545' \
--header 'Content-Type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "admin_peers",
    "params": []
}'
```

Since we are talking to the RPC endpoint node, we will see that the peer it is connecting to is the bootnode (the enode value is the same as bootnodes parameters in docker-compose.yml). An example is shown in Fig. 1.

```
MacPro15:ethereum lma$ curl --location --request POST 'localhost:8545' \
> --header 'Content-Type: application/json' \
> --data-raw '{
>     "jsonrpc": "2.0",
>     "id": 1,
>     "method": "admin_peers",
>     "params": []
> }'
{"jsonrpc":"2.0","id":1,"result":[{"enode":"enode://af22c29c316ad069cf48a09a4ad5cf04a251b411e45098888d114c
20d5953738762afa13711d4ffb3b19aa5de772d8af72f851f7e9c5b164a@172.16.254.2:30303","id":"f19e412334ea6bbcbb22
7a0d6cd1d7966ad489c6fd18c4b3d","name":"Geth/v1.10.1-stable-c2d2f4ed/linux-amd64/go1.16","caps":["eth/64","
","snap/1"],"network":{"localAddress":"172.16.254.4:47368","remoteAddress":"172.16.254.2:30303","inbound":
:false,"static":false},"protocols":{"eth":{"version":66,"difficulty":1,"head":"0xc8d25773460b08854eb45bd3d
f617207f20f5acf900e1b219"},"snap":{"version":1}}}]}
```

Figure 1: Check node connectivity.

Then, get the latest block number of the blockchain.

```
curl --location --request POST 'localhost:8545' \
--header 'Content-Type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 2,
    "method": "eth_blockNumber",
    "params": []
}'
```

After running the nodes for a while, we should see the block number greater than 0x0, which means our miner node already created other blocks after the genesis block. An example is shown in Fig. 2. Note: The miner node takes several minutes to initialize before it starts mining. However if the

```
MacPro15:ethereum lma$ curl --location --request POST 'localhost:8545' --header 'Content-Type: application/json'
aw '{
    "jsonrpc": "2.0",
    "id": 2,
    "method": "eth_blockNumber",
    "params": []
}'
{"jsonrpc":"2.0","id":2,"result":"0x14"}
```

Figure 2: Check block number.

miner has already been running for some time but still get the 0x0 as the latest block number, we should go check the log of the miner node.

Next, we will get the address of our primary account created while the image was built.

```
curl --location --request POST 'localhost:8545' \
```

```
--header 'Content-Type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 3,
    "method": "eth_accounts",
    "params": []
}'
```

The response will contain a list of the created accounts. In our case, there should be only one account, which is "0x65bd095feede84c4fdbbe705dd7e39f2c190b392" as shown in Fig. 3. Note that your address will be different than the example.

```
MacPro15:ethereum lma$ curl --location --request POST 'localhost:8545' \
> --header 'Content-Type: application/json' \
> --data-raw '{
>     "jsonrpc": "2.0",
>     "id": 3,
>     "method": "eth_accounts",
>     "params": []
> }'
{"jsonrpc":"2.0","id":3,"result":["0x65bd095feede84c4fdbbe705dd7e39f2c190b392"]}
```

Figure 3: Primary account number.

Now, check how many Ethereum coins in our account by sending the address of our created account as a parameter as follows. Remember to replace the address to your primary account address.

```
curl --location --request POST 'localhost:8545' \
--header 'Content-Type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 4,
    "method": "eth_getBalance",
    "params": [
        "your-primary-account-address",
        "latest"
    ]
}'
```

If the latest block number of our private blockchain is greater than 0x0, we should see non-zero balance in the account. As this first account we created is, by default, the one receives the mining rewards. An example is shown in Fig. 4. The balance is in hexadecimal format and in wei unit. Wei is the smallest denomination of ether, the currency used to facilitate transactional operations on the Ethereum blockchain network, where 10^{18} wei is equivalent to one ether.

```
MacPro15:ethereum lma$ curl --location --request POST 'localhost:8545' \
> --header 'Content-Type: application/json' \
> --data-raw '{
>     "jsonrpc": "2.0",
>     "id": 4,
>     "method": "eth_getBalance",
>     "params": [
>         "0x65bd095feede84c4fdbbe705dd7e39f2c190b392",
>         "latest"
>     ]
> }'
{"jsonrpc":"2.0","id":4,"result":"0x7439fa2099e580000"}
```

Figure 4: Primary account balance.

2.0.1 Provide a screenshot of your block number after running your Ethereum blockchain for 5 minutes.

Note: The miner node takes several minutes to initialize before it starts mining. However if the miner has already been running for some time but still get the 0x0 as the latest block number, we should go check the log of the miner node.

3 Ethereum Coin Transfer

To test transferring some funds, we will create another account to be the recipient as follows. To create an account, we have to specify the password (in our example, the password is "secret") as the parameter. This password is required to access the account later. If success, we will get the address of the newly created account in return. An example is shown in Fig. 5.

```
curl --location --request POST 'http://localhost:8545' \
--header 'Content-type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 5,
    "method": "personal_newAccount",
    "params": [
        "secret"
    ]
}'
```

The newly created account address in our example is 0x0ebb...3a5, and its password is "secret".

Before sending a transaction, we must make sure that the sender account is unlocked. We will use our primary account as sender as it receives fund from mining rewards. The password used to unlock here must be the same as the one we use to create the account. If the account is unlocked successfully, it returns true, as shown in Fig. 6.

```
MacPro15:ethereum lma$ curl --location --request POST 'http://localhost:8545' \
> --header 'Content-type: application/json' \
> --data-raw '{
>     "jsonrpc": "2.0",
>     "id": 5,
>     "method": "personal_newAccount",
>     "params": [
>         "secret"
>     ]
> }'
{"jsonrpc":"2.0","id":5,"result":"0x0ebb19438d0def9f4dcb77577725434d8625a3a5"}
```

Figure 5: Second account creation.

```
curl --location --request POST 'http://localhost:8545' \
--header 'Content-type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 6,
    "method": "personal_unlockAccount",
    "params": [
        "your-primary-account-address",
        "secret"
    ]
}'
```

```
MacPro15:ethereum lma$ curl --location --request POST 'http://localhost:8545' \
> --header 'Content-type: application/json' \
> --data-raw '{
>     "jsonrpc": "2.0",
>     "id": 6,
>     "method": "personal_unlockAccount",
>     "params": [
>         "0x65bd095feede84c4fdbbe705dd7e39f2c190b392",
>         "5uper53cr3t"
>     ]
> }'
{"jsonrpc":"2.0","id":6,"result":true}
```

Figure 6: Unlock the primary account.

Now we are ready to transfer some fund. In the following example, we are sending 0xf4240 wei (equal to 1 ether) from the primary account 0x65bd...392 to the secondary account 0x0ebb...3a5.

```
curl --location --request POST 'localhost:8545' \
--header 'Content-Type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 7,
    "method": "eth_sendTransaction",
    "params": [
```

```
            {
                "from": "your-primary-account-address",
                "to": "your-second-account-address",
                "value": "0xf4240"
            }
        ]
    }'
```

If the transaction is successfully submitted, we will get the transaction hash (e.g., 0xdb84...c23) in return, as shown in Fig. 7. You can get the status of a transaction by its hash in the future.

```
MacPro15:ethereum lma$ curl --location --request POST 'localhost:8545' --header 'Content-Type: application/json' --data-r
aw '{
    "jsonrpc": "2.0",
    "id": 7,
    "method": "eth_sendTransaction",
    "params": [
        {
            "from": "0x65bd095feede84c4fdbbe705dd7e39f2c190b392",
            "to": "0x0ebb19438d0def9f4dcb77577725434d8625a3a5",
            "value": "0xf4240"
        }
    ]
}'
{"jsonrpc":"2.0","id":7,"result":"0xdb842210062b49d2ebeea320ac24bc763792dcc021cd11268b06bcbe7fc7cc23"}
```

Figure 7: Ether transfer from primary account to second account.

Lastly, verify if we get the fund in the recipient's account as follows.

```
curl --location --request POST 'localhost:8545' \
--header 'Content-Type: application/json' \
--data-raw '{
    "jsonrpc": "2.0",
    "id": 9,
    "method": "eth_getBalance",
    "params": [
        "your-primary-account-address",
        "latest"
    ]
}'
```

3.0.1 Create a third account and transfer one half ether from the second account to the third account. Show necessary screenshots.

3.0.2 Modify the docker-compose.yml file to create a private Ethereum of yours. You will need to create at least three different users/accounts. Demonstrate coin transfer among these accounts.

```
MacPro15:ethereum lma$ curl --location --request POST 'localhost:8545' \
> --header 'Content-Type: application/json' \
> --data-raw '{
>     "jsonrpc": "2.0",
>     "id": 9,
>     "method": "eth_getBalance",
>     "params": [
>         "0x0ebb19438d0def9f4dcb77577725434d8625a3a5",
>         "latest"
>     ]
> }'
{"jsonrpc":"2.0","id":9,"result":"0xf4240"}
```

Figure 8: Transfer verification.